Technical report — `AirPatch`

A deeper write-up than the README: what we built, how it works, and what the numbers say.

- **Track:** Track 2B — Own Project (AirPatch)
- **Event:** Online
- **Team:** AhaWissenschaft AirPatch — William Kannenberg
- **Demo:** https://github.com/AhaWissenschaft/airpatch/blob/main/track_2b/demo/airpatch_demo.mp4 (90 s, English subtitles)
- **Repository:** https://github.com/AhaWissenschaft/airpatch

> **Status of this report.** Every number below was measured. The Apertus results come from one live run per model on 2026-10-05 against the CSCS inference endpoint, after the analysis plan was frozen (commit `0515deb`). All model answers are recorded (`data/cassettes/`), so every number can be re-checked offline (REPLAY).

1. Summary

Organisations that want to use a sovereign model for software work need to know whether it actually produces working code - and they need to find out without sending code to a third party and without trusting the model's own claims. AirPatch gives Apertus small, newly written programming tasks (a specification, a faulty program and a few examples) and asks for a corrected program. A deterministic pipeline that never consults the model then runs every proposal in isolated, offline containers against hidden test cases and records whether it builds, which cases pass, run time, reproducible error fingerprints, model calls and tokens, side by side with model-free baselines. The same pipeline feeds the verifier's findings back to the model for repair rounds, without revealing hidden data. Headline: the measuring instrument is validated (51/51 tasks valid, deterministic, isolation tested with a positive control). On 40 new test tasks Apertus v1.5 solved few: **70B and 8B each 2/40 with one answer (95% CI 1-16%)**, 3/40 with any of three answers, and verifier feedback repaired **none** (0 of 76 repair answers per model passed; most repeated the previous answer verbatim). None of the seven pre-registered comparisons is distinguishable. This is a weak, negative result, reported as measured; what AirPatch contributes is that it is measured reproducibly, offline and without trusting the model.

2. Architecture

Seven small steps, each in its own container with only the mounts it needs (diagram and full table: `docs/ARCHITECTURE.md`):

1. **propose** - builds the prompt from the public task files and asks the model (LIVE), loads a recorded answer (REPLAY) or asks a local stub (MOCK). It never runs code and never sees hidden data.
2. **apply** - turns the answer into a workspace. Only files listed as editable may change; `..`, absolute paths and duplicates are refused.
3. **plan** - lists which candidate runs on which case.
4. **execute** - **one container per hidden test case**: plain Python image without harness code, no network, read-only file system, no capabilities, unprivileged user, CPU / memory / process / output limits; the candidate receives only its own workspace (read-only) and this case's input on stdin.
5. **verify** - the only step that reads expected outputs; assigns verdicts.
6. **feedback** - for repair rounds: counts for hidden cases, details only for the visible examples that are already in the prompt.
7. **report** - Markdown, JSON and a static, offline HTML viewer.

Baselines (no change, mutation search, reference solution, memorising the visible examples) are ordinary candidates and pass through the same path. The host launcher is a POSIX shell script that only starts containers and moves files; no container gets the Docker socket.

Target architecture (mandatory)

b) Air-gapped. At run time the verification pipeline uses no network at all (`--network none` for every step; the MOCK stub runs on an internal Docker network without a route out). The only network use is the propose step talking to the model endpoint, which can be a local OpenAI-compatible server serving Apertus on the same isolated network. The code also fits a) on-premise and c) a Swiss sovereign cloud unchanged, since only `LLM_BASE_URL` differs.

	build time	run time
dependencies	Docker Engine; pinned `python:3.12-slim` image (once)	Docker Engine, POSIX shell, model endpoint
network	image pull (or `docker load` of `make bundle`, 47 MB)	none, except propose -> model endpoint in LIVE mode
packages	none (Python standard library only)	none

3. Use of Apertus

- **Model:** `swiss-ai/Apertus-v1.5-8B` and `swiss-ai/Apertus-v1.5-70B`
- **How it is used:** inference (code repair), evaluated by the pipeline; agents / tool use is deliberately not used: the model writes text only.
- **Where it runs:** CSCS inference endpoint provided for the hackathon (`https://api.inference.cscs.ch/v1`, model ids `swiss-ai/Apertus-v1.5-70B` and `swiss-ai/Apertus-v1.5-8B`; the served model id is recorded with every answer and matched in all 472 answers); any OpenAI-compatible local server works the same way.

Prompt (`src/airpatch/propose.py`): a short system message; the task specification, the editable files with their current content, the visible examples, and the answer format (`FILE: <path>` followed by the complete file in a code block). C0 uses temperature 0, C1 three answers at 0.7, C2 up to two repair rounds at 0. Answers are limited to 2048 tokens. No fine-tuning, no adapters. No other model is used anywhere (no LLM judge): correctness is decided by running tests.

Why a model here, and only here: turning a natural-language specification into a correct change needs understanding; checking the change does not, so that part is deterministic code. The mutation-search baseline (B1) checks whether a task could be solved without a model at all: on all 51 tasks it cannot.

4. Data

- **Tasks:** 51 black-box Python tasks written for this project (11 dev, 40 test; 16 T1, 21 T2, 14 T3) in four areas: text processing, numbers, data structures and simulations, records/time/business rules (including Swiss conventions such as 0.05 rounding, AHV check digits with fictitious numbers, dd.mm.yyyy dates). 562 hidden cases, 79 visible examples, 102 near-miss variants. No task comes from a public benchmark or online judge.
- **Expected outputs** are produced by each task's reference solution.
- **Licence:** CDLA-Permissive-2.0 (data), Apache-2.0 (code). No personal data; all names and numbers are invented.
- Tasks were written with AI assistance (see `docs/PROVENANCE.md`) and accepted only after automatic validation (5.1).

5. Evaluation

5.1 Is the measuring instrument valid? (no model)

check	result
reference solution passes every hidden case	51 / 51 tasks
unchanged start code rejected	51 / 51
program memorising the visible examples rejected	51 / 51
near-miss variants rejected	102 / 102
mutation-search baseline (B1) solves the task	0 / 51
reference solution in the two live runs	40/40 (8B run); 39/40 (70B run: one case of `league` took 5.1 s instead of about 0.1 s once and hit the 5 s limit; 1 of about 9,900 case executions in the two runs; no model answer affected; 40/40 when the 70B answers are replayed)
same answers checked twice give identical verdicts, case results and error fingerprints (E1)	98 candidates, 841 case results, 0 differences (a runaway program may hit the time or the memory limit first depending on machine speed; both count as "limit exceeded" and are reported: 2 such cases in 1 of 3 repeated comparisons)

5.2 Can the instrument be fooled or leak? (no model)

check	result
probe program as candidate searches all readable files for markers that exist only in hidden inputs, expected outputs and verifier source; tries to write and to reach the network	nothing found on 3/3 cases
positive control: same probe with hidden data and harness code deliberately mounted	every marker found
answer writing to `../../hidden/...` and `/etc/passwd`	refused before anything is written (`POLICY_VIOLATION`)
task, hidden, stub and harness files before vs after the run	identical (SHA-256)
repair-round feedback containing any hidden input or expected output	0 in all feedback files; a planted leak is detected
API key in any artefact (sentinel key in tests; real key after the live runs: runs, logs, cassettes, git history)	0 / 0
launcher configuration (explicit network everywhere, only LIVE propose reaches a network, no privileged options)	6 / 6 static checks

5.3 What does Apertus achieve?

Test split, 40 tasks, one run per model (C0 + C1 + C2, 236 calls each, 0 failed calls, 0 tasks excluded as INFRA_ERROR). Solved = all hidden cases pass. 95% Wilson intervals (`results/comparison.md`).

Setup	Apertus 8B	Apertus 70B
C0, one answer (T = 0)	2 / 40 (1-16%)	2 / 40 (1-16%)
C1, any of 3 answers (T = 0.7)	3 / 40 (3-20%)	3 / 40 (3-20%)
C2, C0 + up to 2 repair rounds	2 / 40 (1-16%)	2 / 40 (1-16%)
B0 no change / B1 mutation search / B3 memoriser	0 / 40 each	0 / 40 each
B2 reference solution	40 / 40	39 / 40 (see 5.1)

Pre-registered paired comparisons (exact McNemar, two-sided, `docs/ANALYSIS_PLAN.md`); "only first / only second" = tasks solved by one side only:

comparison	only first	only second	p	reading
70B C0 vs B1	2	0	0.50	not distinguishable
C1 vs C0 (8B; 70B)	1; 1	0; 0	1.00; 1.00	not distinguishable
C2 vs C0 (8B; 70B)	0; 0	0; 0	1.00; 1.00	not distinguishable
8B vs 70B, C0	1	1	1.00	not distinguishable
8B vs 70B, C2	1	1	1.00	not distinguishable

H4 (70B C0 beats B1) is not supported; H5 (C2 >= C0) and H6 (70B >= 8B) hold only as equality - the repair rounds added nothing and the two sizes solved the same number of tasks (different tasks: 70B `age-groups`, `maze`; 8B `maze`, `shift-hours`). With 40 tasks, $p < 0.05$ needs at least 6 tasks (15 points) solved by one side only and none by the other.

Descriptive (not pre-registered, `results/usage.md`): all 236 answers per model were parsed (0 malformed, 0 policy violations, 0 failed calls; 2 build errors each); failures are almost all wrong output (70B: 1,181 failed hidden cases by wrong output, 368 by non-zero exit). The answers are partly right: a C0 answer passes on average 45% (70B) / 37% (8B) of the hidden cases, against 35% for the unchanged start code. Feedback was largely ignored: 49 (70B) and 64 (8B) of 76 repair answers repeated the previous answer verbatim, although the feedback showed the failing visible example with the expected and the actual output.

5.4 Cost and scale

measure	value
hidden-case containers	about 0.15-0.25 s each including start-up; 4 in parallel on 4 vCPUs
`make validate` (51 tasks, 6 candidates each, about 3,400 containers)	7.6 min on 4 vCPUs
`make run` without a key (MOCK, 11 tasks, all conditions)	2.2 min
model calls per task for the full experiment	1 (C0) + 3 (C1) + up to 2 (C2)
model calls per run (40 tasks, cap 240)	236 (40 C0 + 120 C1 + 76 C2)
reported tokens per run, 70B / 8B (cap 3,000,000)	313,667 / 299,256 (about 1,300 per call; prompts 234k / 232k)
model latency per call, median / p90 / max, 70B	5.0 / 9.0 / 12.9 s
model latency per call, median / p90 / max, 8B	1.9 / 3.0 / 4.1 s
full live run (8B: 236 calls, about 4,900 case containers)	16.7 min on 4 vCPUs, 4 parallel calls
code size	2,419 lines (24 modules) + 556-line launcher + 1,144 lines of tests (89 tests)

Scaling: tasks are independent; case containers parallelise (`AIRPATCH_JOBS`) and model calls too (`AIRPATCH_MODEL_JOBS`). Model calls are counted before they are sent (hard cap). For the token cap each call first reserves an upper bound (prompt bytes + template margin + answer limit), so parallel calls cannot together pass the cap; this holds if the endpoint keeps to `max_tokens`. A call that reports more than it reserved is counted, logged as an overrun and stops further calls (unit-tested, including 8 parallel workers). HTTP retries of one call (at most 2) are not extra calls.

6. Limitations

- Tasks are small, single-file Python programs written by us (with AI assistance); results do not transfer to large real-world repositories.
- 40 test tasks give wide intervals (about +/-15 percentage points at 50%); only large differences are distinguishable.
- Isolation relies on Docker and the host kernel; inside a candidate container the program could interfere with its `timeout` process (an outer guard and the verifier's output comparison limit the effect).
- Run times include container start-up.
- Not tested with locally hosted Apertus weights (no GPU available to us); air-gapped operation with a local endpoint is by design, not demonstrated.
- The hidden cases are public in this repository; they are hidden from the model during a run only.
- One prompt, one answer format (complete files), no prompt tuning on the test split (by design). The low solve rate may partly reflect this setup and the strict output formats of the tasks; we do not know how other prompts would do and claim nothing about Apertus beyond these runs.
- One run per model. The 70B run was interrupted by a container restart after its C0/C1 answers were saved and was resumed without asking again (dated deviation in `docs/ANALYSIS_PLAN.md`).
- A post-evaluation adversarial audit identified a defect in partial-run resumption: resuming an interrupted stage treated it as complete as soon as any one task had its answer, so unfinished tasks could be dropped. The recorded 8B and 70B evaluations were independently checked and were not affected, because all required task outputs were present (70B: all 160 C0/C1 answers were written before the interruption; 8B was never resumed). The defect was repaired only after the scientific evaluation had been frozen; no model responses or reported evaluation results were regenerated.
- Endpoint behaviour (model version behind an id, sampling) is outside our control; served model ids and all answers are recorded.

7. Reproducibility

- Hardware used: 4 vCPU, 16 GB RAM, Linux, Docker 29; no GPU.
- `make run` from `track_2b/` on a clean checkout. Without a key it re-checks the recorded Apertus 70B answers (**REPLAY**, labelled as such; `AIRPATCH_CASSETTE=apertus-8b-20261005` for 8B); with `LLM_NAME`, `LLM_BASE_URL`, `LLM_API_KEY` it asks the model (**LIVE**). The 8B vs 70B analysis: `make compare` on two REPLAY runs.
- All checks: `make test validate regress determinism security`.
- Seeds: none needed (temperature 0 for C0/C2; C1 answers are recorded).

- Frozen analysis plan: commit `0515deb`. Exact commit of code, data and results: `8bb67bc`. Later commits are post-evaluation hardening (see `docs/POST_EVALUATION_AUDIT.md`): text, tests, the launcher, the resume path of the propose step and one connection-release line in the API client. Prompt construction, parser, verifier, feedback, tasks, hidden cases, recorded answers and analysis are unchanged; the recorded prompt hashes of all 320 C0/C1 answers still match the current code. No material discrepancy affecting the reported frozen evaluation was found in the reviewed areas.
- Check on a fresh clone of `8bb67bc` without a key: all gates passed (`make run`, `test` 60/60, `validate` 51/51, `regress`, `security`, `determinism` 0 differences), and REPLAY of both cassettes reproduced all 472 model verdicts and every number in `results/comparison.md` and `results/usage.md`, apart from the mode label, the empty call ledger of a replay and the `league` oracle case (which passed).
- The public repository is a single snapshot of the final state; the commit identifiers above refer to the development history (see the README).

8. Next steps

Multi-file tasks and a compiled language (real build step); more tasks from public-sector code bases with permissive licences; running the same experiment against locally served Apertus weights on-premise.

License

Creative Commons Attribution 4.0 (CC-BY-4.0). All HackApertus projects are open-sourced.

References

- Hack Apertus project template: https://github.com/hackapertus/project-template
- Apertus v1.5 models: https://huggingface.co/swiss-ai
- Wilson, E. B. (1927). Probable inference, the law of succession, and statistical inference.
- McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions.